

5. Sistema Comercial Integral: POS, Cotizaciones, Ventas y Facturación Simulada

Equipo:

- ❖ Paredes Dominguez Jassiel
- ❖ Torres Galván Alejandro Aldahir
- ❖ Aguilar García Angel Gabriel
- ❖ Bertadillo Villalobos Leobardo Daniel
- ❖ Arias Hernández José

Objetivo

Crear un sistema comercial para administrar productos, clientes, cotizaciones, ventas, caja, comprobantes y reportes.

Roles sugeridos

Rol	Descripción
Administrador	Configura productos, precios, usuarios y reportes.
Vendedor	Crea ventas y cotizaciones.
Cajero	Cobra ventas y realiza cortes.
Supervisor	Autoriza descuentos, cancelaciones o ajustes.

Módulos principales

Módulo	Descripción
Productos/servicios	Catálogo de venta.
Clientes	Datos de clientes frecuentes.
Cotizaciones	Generación de propuestas comerciales.
Ventas POS	Venta rápida directa.
Caja	Apertura, cierre, ingresos y egresos.
Facturación simulada	Comprobante PDF no fiscal.

Reportes	Ventas, productos, cotizaciones, cortes.
----------	--

Privilegios dinámicos esperados

Privilegio	Ejemplo
Crear venta	Permite vender desde POS.
Aplicar descuento	Solo usuarios autorizados.
Cancelar venta	Requiere privilegio especial.
Abrir caja	Permite iniciar operación diaria.
Cerrar caja	Permite hacer corte.
Ver utilidad	Solo supervisores o administradores.
Generar factura simulada	Permite emitir comprobante.

Must have

- Productos/servicios.
- Clientes.
- Cotizaciones.
- Conversión de cotización a venta.
- POS.
- Caja.
- Comprobante PDF.
- Reportes comerciales.
- Compras.

Extra avanzado

- Folios automáticos.
- Descuentos por producto o venta.
- Pagos mixtos.
- Historial de cliente.

- Alertas stock bajo y proximo a caducar

Base obligatoria para todos los equipos

Todos los proyectos deben usar:

Categoría	Requisito obligatorio
Arquitectura general	SOFEA
Frontend	Microfrontends como componentes/aplicaciones separadas
Backend	Microservicios
Frontend framework	Next.js con TypeScript
Backend framework	NestJS
Base de datos	PostgreSQL
Documentación API	OpenAPI / Swagger / Scalar / ApiDog
Seguridad	Autenticación y autorización
Permisos	Multirol y multiprivilegio
Implementación	Al menos un módulo funcional completo
Repositorio	Estructura clara, ramas y README

Entregables según metodología

Shape Up

Si el equipo elige Shape Up, deberían entregar:

Artefacto	Qué revisar
Pitch	Problema, solución, alcance y valor
Appetite	Tiempo máximo que dedicarán al módulo
Rabbit holes	Riesgos o partes que podrían complicarse
No-gos	Qué no se va a construir

Scopes	División del trabajo por partes funcionales
Hill Chart	Avance del descubrimiento a la ejecución
Circuit breaker	Qué harán si no terminan a tiempo
Implementación	Módulo terminado dentro del alcance definido

Ejemplo de pitch:

Problema:

Los usuarios no pueden consultar de forma clara el estado de sus pedidos.

Solución:

Crear un microfrontend de pedidos conectado a un microservicio de órdenes.

Appetite:

2 semanas.

No-gos:

No se implementará pago en línea ni facturación real.

Rabbit holes:

Sincronización entre inventario y pedidos.

Módulo de referencia:

Consulta y actualización de estado de pedidos.

Definir puntos de arquitectura obligatoria del proyecto

Microfrontends

Deben explicar:

Punto	Pregunta guía
App Shell	¿Cuál es el contenedor principal del frontend?
Microfrontends	¿Qué módulos frontend existen?
Comunicación	¿Cómo consumen APIs?
Estado	¿Qué estado es local y qué estado es global?
Permisos	¿Cómo se muestran/ocultan funciones por privilegio?
Integración visual	¿Cómo mantienen diseño consistente?

Microservicios

Deben explicar:

Punto	Pregunta guía
Servicios	¿Qué microservicios existen?
Responsabilidad	¿Qué hace cada servicio?
Comunicación	¿REST, eventos o ambos?
Datos	¿Qué tablas o esquemas usa cada servicio?
Seguridad	¿Cómo se protegen los endpoints?
Errores	¿Cómo manejan fallos entre servicios?

Patrones y Principios

Cada equipo debe explicar qué patrones aplicó, dónde los aplicó y por qué tienen sentido en su proyecto, los **principios SOLID son obligatorios**.

Para patrones de diseño se verán en clase pero pueden revisarl la [documentación oficial](#).

Tecnologías obligatorias y opcionales

Obligatorias

Área	Tecnología
Frontend	Next.js + TypeScript + MUI
Backend	NestJS (Prisma / TypeORM)
Base de datos	PostgreSQL
Arquitectura frontend	SOFEA + Microfrontends
Arquitectura backend	Microservicios
API Docs	OpenAPI / Swagger / Scalar / ApiDog
Seguridad	JWT, roles y privilegios
Repositorio	Monorepo o estructura multiapp ordenada

Una recomendación del scaffolding monorepo podría ser:

proyecto/

├─ README.md

├─ LICENSE

- ├ .gitignore
- ├ .editorconfig
- ├ .gitattributes
- ├ .nvmrc
- ├ .npmrc
- ├ pnpm-workspace.yaml
- ├ package.json
- ├ turbo.json
- ├ tsconfig.base.json
- ├ biome.json
- ├ commitlint.config.cjs
- ├ .husky/
 - | └ pre-commit
 - | └ commit-msg
- ├ docs/
 - | └ 00-overview/
 - | | └ vision.md
 - | | └ principles.md
 - | | └ glossary.md
 - | └ 01-architecture/
 - | | └ c4/
 - | | | └ context.md
 - | | | └ container.md
 - | | | └ component.md
 - | | | └ deployment.md
 - | | └ decisions/
 - | | | └ ADR-0001-tenant-resolution.md
 - | | | └ ADR-0002-auth-oidc-jwks.md
 - | | | └ ADR-0003-eventing-kafka-outbox.md
 - | | └ standards/

- | | └─ module-contract.md
- | | └─ api-guidelines.md
- | | └─ security-baseline.md
- | | └─ observability-baseline.md
- | | └─ branching-conventions.md
- | └─ 02-api/
 - | | └─ openapi/
 - | | | └─ gateway.yaml
 - | | | └─ services/
 - | | └─ postman/
 - | | | └─ collections/
 - | | | └─ environments/
- | └─ 03-runbooks/
 - | | └─ local-dev.md
 - | | └─ qa-deploy.md
 - | | └─ incident-response.md
- | └─ 04-product/
 - | | └─ figma-links.md
 - | | └─ roadmap.md
 - | | └─ tenants-strategy.md
- |
- └─ infra/
 - | └─ docker/
 - | | └─ compose/
 - | | | └─ docker-compose.dev.yml
 - | | | └─ docker-compose.qa.yml
 - | | | └─ docker-compose.prod.yml
 - | | └─ images/
 - | | | └─ gateway/
 - | | | └─ services/

| | | └ observability/

| | └ scripts/

| | └ up.sh

| | └ down.sh

| | └ reset-db.sh

| └ k8s/

| | └ base/

| | └ overlays/

| | | └ dev/

| | | └ qa/

| | | └ prod/

| | └ helm/

| └ ci/

| | └ jenkins/

| | | └ Jenkinsfile

| | | └ pipelines/

| | | └ shared-libs/

| | └ scripts/

| | └ build.sh

| | └ test.sh

| | └ scan.sh

| └ observability/

| └ otel/

| └ grafana/

| └ prometheus/

| └ dashboards/

|

└ agents/ # IA para cumplir guía (prompts + reglas + plantillas) o gsd

| └ README.md

| └ rules/

- | | |─ architecture-rules.md
- | | |─ security-rules.md
- | | |─ api-rules.md
- | | |─ frontend-rules.md
- | | |─ backend-rules.md
- | | └─ mobile-rules.md
- | |─ checklists/
- | | |─ pr-checklist.md
- | | |─ release-checklist.md
- | | |─ module-readiness-checklist.md
- | | └─ threat-model-lite.md
- | |─ prompts/
- | | |─ cursor/
- | | | |─ scan-changes-for-adr-compliance.md
- | | | |─ generate-module-skeleton.md
- | | | └─ enforce-module-contract.md
- | | |─ chatgpt/
- | | | |─ design-c4-diagrams.md
- | | | |─ create-openapi-first.md
- | | | └─ generate-tests.md
- | | └─ sonar/
- | | └─ fix-security-findings.md
- | └─ templates/
- | |─ ADR-template.md
- | |─ module-contract-template.md
- | |─ openapi-template.yaml
- | └─ README-module-template.md
- |
- |─ apps/
- | |─ frontend/

```

| | └─ web-shell/          # Next.js host
| | └─ microfrontends/
| | | └─ projects-mf/
| | | └─ billing-mf/
| | | └─ cards-mf/
| | └─ commons/            # commons SOLO frontend (Atomic + UI + utils)
| |   └─ ui/
| |   └─ hooks/
| |   └─ state/
| |   └─ api/
| |   └─ utils/
| |
| └─ backend/
| | └─ gateway/            # API Gateway
| | └─ services/
| | | └─ auth-service/
| | | └─ tenant-service/
| | | └─ projects-service/
| | | └─ billing-service/
| | | └─ cards-service/
| | └─ commons/            # commons SOLO backend (lib compartida)
| |   └─ contracts/        # DTOs/Events versionados
| |   └─ observability/    # logger, tracing, metrics
| |   └─ security/         # jwks client, auth guards, crypto
| |   └─ utils/
| └─
|
└─ packages/              # Shared cross-domain (opcional y explícito)
  └─ shared-schemas/      # JSON Schema / OpenAPI fragments
  └─ shared-protos/        # si usas gRPC

```

```

└─ shared-devtools/          # configs, lint rules, generators
└─
└─ scripts/
    └─ bootstrap.sh          # instala + prepara repo
    └─ dev.sh                # levanta dev (compose + apps)
    └─ lint.sh
    └─ test.sh
    └─ release.sh

```

Recomendadas

Área	Tecnología / práctica
Estado	Redux Toolkit / RTK Query
Paquetes	pnpm
Contenedores	Docker / Docker Compose
Documentación	Obsidian, Miro, diagrams.net
Testing API	Postman / Insomnia
Calidad	SonarQube
Loggers	Nativo / Grafana Loki / Pino / Elastic Stack

Avanzadas

Área	Tecnología / práctica
Eventos	Kafka
Resiliencia	Circuit Breaker
Procesos distribuidos	Saga Pattern
Cache	Redis
Seguridad avanzada	JWKS, refresh token rotation
CI/CD	Jenkins
Tenancy	Schema-per-tenant, sharding o DB dedicada

Estrategia de ramas

El equipo deberá documentar su estrategia de ramas respondiendo:

Pregunta guía	Qué se espera
¿Cuáles serán sus ramas base?	Por ejemplo: main, dev, qa, release, etc.
¿Qué representa cada rama?	Explicar el propósito de cada una.
¿Cómo se integrarán los cambios?	Directo, Pull Request, revisión por compañero, merges programados, etc.
¿Cómo nombrarán las ramas de trabajo?	Definir formato propio.
¿Qué tipos de ramas usarán?	Funcionalidad, corrección, documentación, pruebas, investigación, etc.
¿Cuándo se elimina una rama?	Al terminar, al integrarse, al cancelar una tarea, etc.
¿Cómo evitarán conflictos?	Actualización frecuente, ramas pequeñas, revisión previa, comunicación, etc.

Requisito transversal para todos los proyectos

Todos los equipos deberán implementar un esquema de **privilegios dinámicos por módulo**.

Must have técnico general

Requisito	Descripción
Módulos configurables	Cada sistema debe identificar sus módulos principales.
Roles funcionales	Deben existir perfiles base de usuario según el negocio.
Privilegios dinámicos	El sistema debe permitir asignar permisos específicos por módulo y acción.
Menús dinámicos	El menú debe mostrar u ocultar opciones según privilegios.
Acciones protegidas	Botones como crear, editar, eliminar, aprobar o exportar deben depender de permisos.
Validación backend	No basta con ocultar botones en frontend; el backend también debe validar permisos.
Dashboard	Cada usuario debe ver información acorde a sus privilegios.

Reportes

La visualización y exportación de reportes debe depender de permisos.

Criterio final común para todos

Para que los proyectos no se queden como CRUDs simples, puedes poner esta regla:

Cada proyecto debe demostrar una plataforma web integral con módulos relacionados entre sí, permisos dinámicos por módulo y acción, dashboards, reportes filtrables, carga o exportación de información, validaciones de negocio, documentación técnica y despliegue funcional.

Y la regla más importante:

No se evaluará únicamente que el usuario tenga un rol, sino que el sistema sea capaz de mostrar, ocultar, permitir o bloquear funcionalidades según los privilegios asignados dinámicamente.